

Lecture 8 Graph(III), Graph(IV)

Shortest Path Algorithm

Weighted path length:

$$\sum_{e_i \in P} w(e_i)$$

Unweighted path length: The number of edges on the path.

Single-Source Shortest-Path Problem

Given as input a weighted graph, and a distinguished vertex, find the shortest weighted path from the vertex to another vertex in the graph. (No negative weights)

Unweighted Shortest Paths

- Create a table, recording the distance from the source to the vertex, the status whether it is checked and the path recording the former vertex.
- Initialize the distance(0), status(1) and path of source, then enqueue the source.
- BFS:
 - When the queue is not empty, dequeue the vertex u .
 - Traverse all neighbors v of u .
 - If v is not visited and can reach, renew the distance as the distance of the former vertex plus 1, the status and the path, enqueue v .

Time Complexity: $O(|V| + |E|)$.

Weighted Shortest Paths: Dijkstra's Algorithm

It is a greedy algorithm.

We assume that there is a path from the source vertex to every other vertex in the graph. If v_k is the second last vertex on the shortest path from the source to v_j , then the path from the source to v_k must be the shortest path.

Let S be the set of vertices whose minimum distance from the source vertex has been found, i.e., $S = \{s \text{ and vertices whose shortest paths have been found}\}$. Initially $S = \{s\}$. For each vertex v not in S , the path from the source to v must only pass the vertices in S , i.e.,

$$\{s \rightarrow (v_i \in S) \rightarrow v\}$$

Iterative process:

- Initialize the distance of all vertices except source as the **direct weight from the source to it(infinite for undirect)**.
- **Choose the nearest vertex** u from source to it, put it into S .
- Renew all vertices v adjacent to u : **if $d(s \rightarrow u \rightarrow v) < d(s \rightarrow v)$, renew the distance of v .**
- Repeat the former two steps until all vertices confirm the shortest path.

Pseudocode:

```
void Dijkstra( Table T )
{ /* T is initialized by Figure 9.30 on p.303 */
  Vertex V, W;
  for ( ; ; ) { /* O( |V| ) */
    V = smallest unknown distance vertex;
    if ( V == NotAVertex )
      break;
    T[ V ].Known = true;
    for ( each W adjacent to V )
      if ( !T[ W ].Known )
        if ( T[ V ].Dist + Cvw < T[ W ].Dist ) {
          Decrease( T[ W ].Dist to
            T[ V ].Dist + Cvw );
          T[ W ].Path = V;
        } /* end-if update W */
    } /* end-for( ; ; ) */
  } /* not work for edge with negative cost */
```

Minimum Spanning Tree

A **spanning tree** of an undirected graph G is a **tree** containing **all vertices $V(G)$** .

Note:

- A spanning tree exists if and only if G is connected.
- The spanning tree has the number of edges $|V| - 1$.
- Adding a non-tree edge to a spanning tree we can obtain a cycle.

For weighted graph, the **weight of a subgraph** is the sum of **the weights of all edges** in the subgraph.

A **minimum spanning tree(MST)** for a weighted graph is a spanning tree **with minimum weight**.

Finding MST of a Graph

Solution to find minimum spanning trees must satisfy:

- Only edges with in the graph
- **Exactly $|V| - 1$ edges.**
- May not use edges that would produce a cycle.

Two efficient well known methods:

- Prim's algorithm
- Kruskal's algorithm

Prim's algorithm

Greedy algorithm, similar to Dijkstra's algorithm.

- Initially choose a vertex into the set of MST
- Find the edge **with minimum weight** connecting the vertex within the tree and outside the tree.
- Put this edge and corresponding vertex into MST, renew adjacent unknown smallest distance vertex. $\text{dist}[w] = \min\{\text{dist}[w], \text{distance from } v \text{ to } w\}$. Here the distance is the vertex to the MST.
- Repeat the former two steps until all vertices are in the set.

pseudocode:

```
void Prim( Table T )
{ /* T is initialized by Figure 9.30 on p.303 */
  Vertex V, W;
  for ( ; ; ) {
    V = smallest unknown distance vertex;
    if ( V == NotAVertex )
      break;
    T[ V ].Known = true;
    for ( each W adjacent to V )
      if ( !T[ W ].Known )
        if ( Cvw < T[ W ].Dist ) {
          T[ W ].Dist = Cvw;
          T[ W ].Path = V;
        } /* end-if update W */
    } /* end-for( ; ; ) */
  }
}
```

Suitable for dense graph. $O(V^2)/O(E \log V)$.

Kruskal's algorithm

Build a minimum spanning tree by adding edges to MST one at a time.

- Select the edges in the graph in nondecreasing order of the weight.
- If it does not form a cycle, add it into MST. If do, discard it.
- Repeat the former two steps until MST has $n - 1$ edges.

More specifically, every vertex are a single set. If an edge connects two vertices in two different sets, it does not form a cycle. Then merge the two sets.

Suitable for sparse graph. $O(E \log E)$.

Network Flow Problems

The network flow problem is used for weighted directed graph. Every weight represents the capacity of the edge. It concentrates on the maximum flow from source to sink.

Total coming in v equals to total going out in v where $v \in (s, t)$.

A Simple Algorithm (Fold-Fulkerson algorithm)

Use two graphs--Flow graph and residual graph. For each edge, the sum of the weight of flow graph and residual graph equals to the original graph's.

- Find any path from source to sink in residual graph.
- Take the minimum edge on this path as the amount of flow and add to flow graph.
- Update the residual graph and remove 0 flow edges.

The algorithm is not completely correct (depending on the sequence of selecting path)
A solution--allow the algorithm to undo its decisions. For each edge in flow graph, add an reverse edge with the same cost as of the path to residual graph.

Note that this algorithm only find maximum flow when the flows are rational numbers.

Application of DFS

Connected components of undirected graphs

Each DFS can forms a DFS tree. The number of DFS trees are the amount of connected components.

Biconnectivity

Biconnected undirected graph:

There are no vertices whose removal disconnects the rest of the graph.

For not-biconnected graph, there exists articulation points that if it is deleted the connected components will increase.

A biconnected component is a maximal biconnected subgraph.

Find the Articulation Points of a Connected Graph

- Use DFS to obtain a spanning tree of G .
- The root is an articulation point if and only if it has at least 2 children.
- Any other vertex is articulation point if and only if the vertex has at least 1 child and there is no path(including back edge) from its child to its ancestor in the tree.

The $num(v)$ is the index of v of the preorder traverse of the tree.

The $low(v)$ is the lowest-numbered vertex which is reachable from v by taking zero or more tree edges (can including one back edge).

$low(u) = \min\{num(u), \min\{low(w), w \text{ is a child of } u\}, \min\{Num(w), (u, w) \text{ is a back edge}\}\}.$

$low(v)$ can be calculated by post order traverse.

Therefore, u is an articulation point if and only if

- u is the root and has at least 2 children or

- u is not the root but has at least 1 child such that $low(child) \geq Num(u)$.

Strongly Connected Component

A directed graph is strongly connected if there is a directed path from v_i to v_j and also from v_j to v_i . And a strongly connected component is a maximal subgraph that is strongly connected.

The steps to find strong components in a digraph are given below:

- Perform DFS(s) on the given graph G to produce a set of DFS spanning trees called a DFS spanning forest.
- All vertices of the graph G are numbered with the help of post-order traversal of DFS spanning forests.
- Obtain the reverse graph G_r , of G by reversing the edges of G in G_r .
- Perform the DFS traversal on Graph G_r , starting at the vertex which is numbered highest and mark the vertex as visited.
- If one traversal procedure does not visit all the vertices of G_1 , perform another DFS traversal starting at the vertex with the highest number among all the remaining vertices.

Euler Circuits

Euler tour Problem: Draw each line exactly once without lifting pen from the paper.

Euler circuit problem: Draw each line exactly once without lifting pen from the paper, and finish at the starting point.

- An Euler circuit is possible only if the graph is connected and each vertex has an even degree.
- An Euler tour is possible if there are exactly two vertices having odd degree. One must start at one of the odd-degree vertices

Algorithm:

- Judge whether the circuit satisfy the conditions.
- DFS from any point.
- If there is no unvisited edges of this point, put the point into path.
- When backtracking, if there is unvisited edges, continue to DFS
- Reverse the path.

$$T = O(|E| + |V|).$$